Memoria PAI-2 BYODSEC (ST-5)

Grupo: ST-5 - Tomás Gutiérrez, Miguel Ridao y Julio Morales

Repositorio: github.com/tomgutgar/PAI2-ST5

1. Resumen

Hemos implementado un gateway VPN tipo *road warrior* para la Universidad U-Secure y el cliente que llevaría cada dispositivo BYOD (portátil, móvil o tablet personal). A diferencia del PAI-1, aquí la protección no la hacemos nosotros a mano: todo el tráfico va dentro de **TLS 1.3 con autenticación mutua (mTLS)**, y encima de eso el usuario tiene que meter su usuario y contraseña. Para que el servidor acepte a un dispositivo, este tiene que presentar un certificado X.509 emitido por nuestra propia **CA corporativa**.

Los objetivos que nos marcamos, siguiendo el enunciado, fueron:

- Montar una PKI con CA propia que emita, revoque y publique en una CRL los certificados del servidor y de los dispositivos
- Aceptar solo TLS 1.3 con suites AEAD y PFS, y exigir certificado de cliente en el túnel
- Login en dos capas (certificado + contraseña), cierre de sesión que destruye el túnel y 300 cuentas precargadas de PDI, PAS e investigadores
- Comprobar que aguanta 300 sesiones a la vez y medir cuánto cuesta TLS frente al mismo servidor sin cifrar
- Dejar evidencias: 4 capturas .pcap, el `audit.log`, 34 tests y 19 ataques simulados que el sistema tiene que bloquear

Igual que en el PAI-1 usamos solo la librería estándar de Python (`ssl`, `socket`, `sqlite3`, `hashlib`, `threading`), con OpenSSL 3 por debajo. La CA se gestiona con la línea de comandos de `openssl` desde un script (`pki/pki.sh`), así que no hay nada que instalar con pip. Las instrucciones de instalación y uso están en el apartado 8.

1.1 Decisiones técnicas

De las tres opciones de la tabla de "Capa de transporte" elegimos **sockets TCP con el módulo `ssl`**. Es la que más se parece a lo que hicimos en el PAI-1 (mensajes JSON de una línea) y nos deja controlar cada paso del handshake, que es justo lo que se evalúa. Con REST o gRPC habríamos metido un framework entero solo para transportar unos pocos comandos.

Decisión	Qué hacemos	Por qué
Versión y suites (RS2)	Solo TLS 1.3. Suites `TLS_AES_256_GCM_SHA384` y `TLS_CHACHA20_POLY1305_SHA256`, grupos `X25519:secp384r1`, todo en `openssl_tls13.cnf`	En Python `set_ciphers()` no afecta a las suites de TLS 1.3 (lo probamos y el servidor seguía aceptando AES-128). La única forma es cargar ese fichero con `OPENSSL_CONF` antes de `import ssl`
Doble comprobación	Además del fichero, `minimum_version = maximum_version = TLSv1_3`; el servidor no arranca si la política no se cargó y revisa versión y suite tras cada handshake	Si alguien cambia el orden de los `import`, la política dejaría de aplicarse sin avisar
PKI (RS1a)	"U-Secure Root CA" RSA 3072 (5 años), certificado de servidor `serverAuth` y de dispositivo `clientAuth` con `CN=usuario` y `OU=rol`, firmados con `openssl ca`	`openssl ca` lleva el `index.txt` y los números de serie, así que la revocación y la CRL funcionan como en una CA de verdad
Clave del dispositivo	La genera el cliente; al servidor solo le llega una CSR. El sujeto del certificado lo pone la CA	La clave privada no sale nunca del dispositivo y el usuario no puede elegir su rol
mTLS (RS1b)	Puerto 7443 con `CERT_REQUIRED` y comprobación de CRL. El cliente valida la cadena y el nombre del servidor	Sin certificado válido el servidor no lee ni un byte de aplicación
Alta (RF1a)	Puerto 8443 (TLS 1.3 sin certificado de cliente) que solo acepta `REGISTER`. Hace falta un código de un solo uso que da el administrador	Un dispositivo nuevo aún no tiene certificado. El código cumple la política: "solo usuarios previamente autorizados"

Inmutabilidad (RF1b)	Triggers `BEFORE UPDATE/DELETE` en SQLite. Re-emitir = revocar + código nuevo + fila nueva	Lo garantiza la base de datos, no el código Python
Contraseñas	PBKDF2-HMAC-SHA256, 600.000 iteraciones, salt de 16 bytes, `compare_digest`	Lo mismo que en el PAI-1, siguiendo OWASP
Bloqueo	5 fallos seguidos bloquean la cuenta 5 min. El intento se cuenta antes de calcular el PBKDF2	Si se contara después, varios login en paralelo podrían probar más de 5 contraseñas
Cierre (RF4)	`LOGOUT` manda `close_notify` y cierra el socket. `num_tickets = 0`	Sin tickets no se puede reanudar la sesión: cada túnel usa claves ECDHE nuevas (PFS)
Concurrencia (RS3)	Un hilo por conexión, `listen(512)`, máximo 400 sesiones en 7443 y 50 en 8443; 10 s para el handshake y 60 s para el login	Sencillo y suficiente para 300 usuarios. Los plazos evitan que alguien ocupe hilos con handshakes a medias
Auditoría (RS4b)	Una línea por evento con IP, CN, serie del certificado, versión y suite. Nunca contraseñas ni códigos	Trazabilidad que pide el ENS y minimización de datos del RGPD/LOPDGDD

Los perfiles de acceso son un diccionario en `server.py`: el PDI accede a correo, docencia, repositorio y bd_investigacion; el PAS a correo, gestión y repositorio; y los investigadores (INV) a correo, repositorio, bd_investigacion y servidores_internos. Los recursos están simulados: devuelven un texto fijo.

2. Diagrama de secuencia

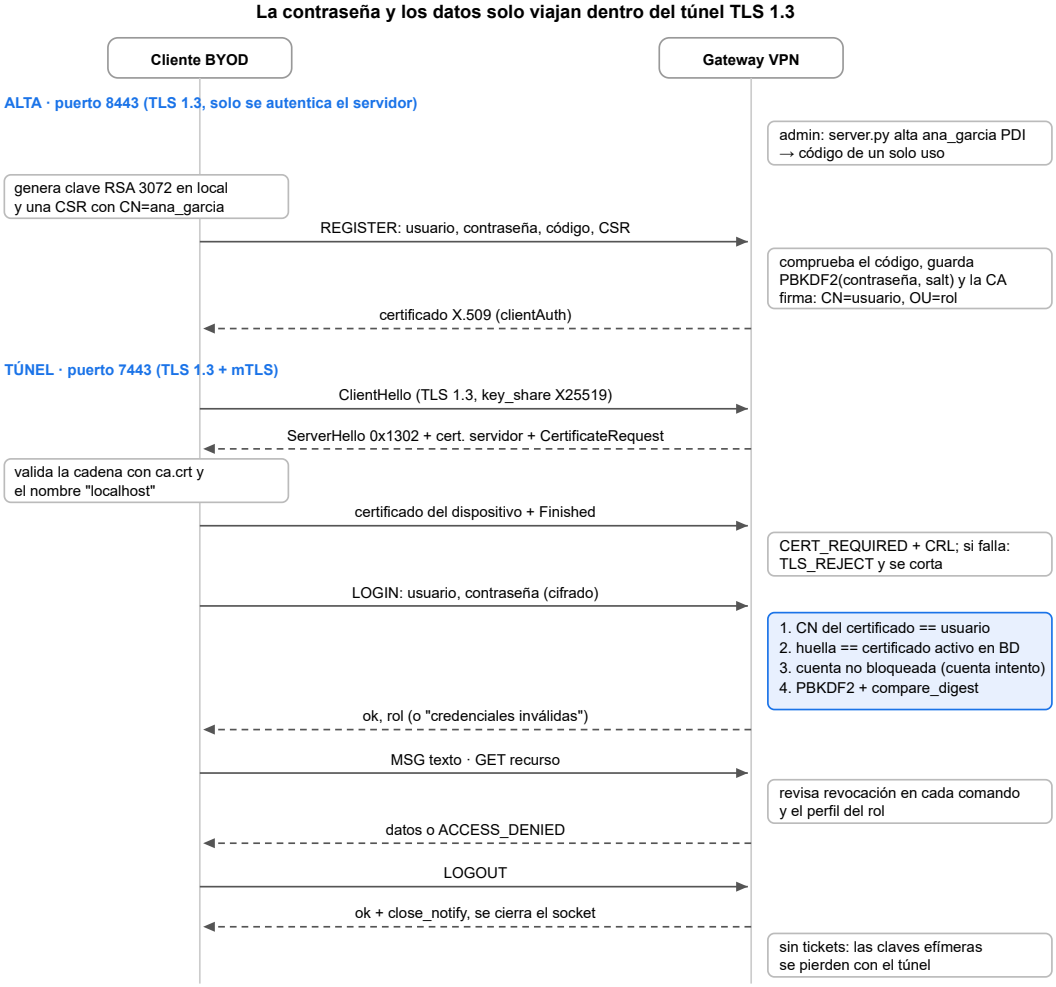


Figura 1: Diagrama de secuencia del alta y de una sesión completa

El alta es la única operación que va por el puerto 8443, porque el dispositivo todavía no tiene certificado; aun así va cifrada con TLS 1.3 y el cliente comprueba que habla con el servidor de verdad. A partir de ahí todo pasa por el 7443, y el orden de las comprobaciones del login está pensado para que solo el dueño del certificado pueda gastar intentos: si alguien prueba con el usuario de otro, falla en el paso 1 y no bloquea esa cuenta.

3. Implementación por módulos

Fichero	Qué hace
src/common.py	Carga `openssl_tls13.cnf` antes que nada, crea los contextos TLS de servidor y cliente (`contexto_servidor`, `contexto_cliente`), el protocolo de una línea JSON (máx. 64 KiB) y la función `audit`, que limpia los valores para que no se puedan inyectar líneas falsas en el log
src/server.py	Clase `Gateway`: escucha en 7443 y 8443, un hilo por conexión con semáforo de 400, `_handshake` con plazo total de 10 s, `_login` (las 4 comprobaciones), `_bucle` (MSG, GET, LOGOUT) y `_cerrar`. Recarga la CRL sola cuando cambia. Por consola: `alta` y `revocar`
src/altas.py	`registrar` y `revocar`: valida la entrada, comprueba que el CN de la CSR coincide con el usuario y llama a `pki.sh` para firmar. Lo usan el servidor y `seed.py`
src/db.py	SQLite con usuarios, certificados, códigos de alta, revocaciones y mensajes, todo con triggers de inmutabilidad. PBKDF2 y hash de los códigos
src/client.py	Menú (registrar, iniciar sesión, mensaje, recurso, cerrar sesión) y modo por comandos para los scripts. Genera la clave y la CSR y traduce los errores de OpenSSL a mensajes entendibles
src/seed.py	RF5: 300 cuentas (pdi_001…100, pas_001…100, inv_001…100) con su certificado
pki/pki.sh	`init`, `server`, `sign`, `revoke` y `crl` sobre `openssl ca`. Solo firma claves RSA de 3072 bits o más y usa `flock` para que dos procesos no firmen a la vez

4. Grado de completitud

4.1 Matriz de trazabilidad

El enunciado pasa de RF2 a RF4, así que respetamos esa numeración (no hay RF3). Todos los requisitos y los cinco objetivos, incluido el extra, están implementados y probados.

Requisito	Fichero y función	Test	Ataque	Evidencia
O1 Canal seguro para credenciales	common.py → contexto_servidor/cliente; server.py → _bucle	test_rf2_flujo_completo_y_dos_capas	A01, A11	pcap/01 y 02, analisis.txt
O2 PKI + TLS 1.3 + mTLS	pki/pki.sh, openssl_tls13.cnf, common.py	test_rs2_negocia_tls13_suite_aead_y_x25519	A02–A06	audit.log (TLS_REJECT)
O3 Rendimiento con 300	bench/benchmark.py, plain_server.py	test_rs3a_300_sesiones_simultaneas	–	benchmark/resultados.csv, resumen.md
O4 Tráfico y ataques	scripts/captura.sh, ataques/ataques.py	test_seguridad (20 tests)	A01–A19	pcap/*, ataques/resultado_*.log
O5 (extra) Interceptación y suplantación	ataques/mitm_proxy.py	tests de A12 a A16	A12–A16	A13_volcado_mitm_pasivo.txt, pcap/04
RF1a Registro con certificado	altas.py → registrar; server.py → atender_alta; client.py → registrar	test_rf1a_registro_emite_certificado, test_rf1a_codigo_invalido_o_reutilizado, test_rf1a_alta_fallida_conserva_la_clave	–	ENROL_OK / ENROL_FAIL
RF1b Inmutabilidad	db.py → ESQUEMA (triggers); altas.py → revocar	test_rf1b_bd_inmutable, test_rf1b_reemision_solo_tras_revocar, test_rf1b_revocar_corta_sesion_abierta	A09, A10	REVOKE, pki/ca/index.txt
RF2 Login en dos capas	server.py → _login, _bucle	test_rf2_flujo_completo…, sin_certificado…, cert_de_otro_usuario, comandos_sin_login, bloqueo_por_fuerza_bruta	A04, A07, A08, A18	LOGIN_FAIL, LOCKOUT, UNAUTH_CMD
RF4 Cierre y desmontaje	server.py → _cerrar; client.py → Tunel.cerrar; num_tickets = 0	test_rf4_logout_destruye_el_tunel	A17	LOGOUT, DISCONNECT
RF5 Usuarios precargados	seed.py; server.py → PERFILES	test_rf5_perfiles_por_rol, test_carga	A18	seed_credentials.csv, ACCESS_DENIED
RS1a CA corporativa	pki/pki.sh, pki/openssl_ca.cnf	tests rf1a y rf1b_reemision	A06, A09	index.txt, crl.pem
RS1b mTLS	common.py → contexto_servidor (CERT_REQUIRED + CRL)	tests de A04, A05, A06, A09, A12	A04–A06, A09, A12	TLS_REJECT CERTIFICATE_VERIFY_FAILED
RS2a Solo TLS 1.3	openssl_tls13.cnf; common.py → _tls13; server.py → _handshake	tests de A02 y A11	A02, A11	UNSUPPORTED_PROTOCOL, pcap/03

RS2b AEAD + PFS	openssl_tls13.cnf (Ciphersuites, Groups)	test de A03, test_rs2_negocia…	A03	NO_SHARED_CIPHER, ServerHello 0x1302
RS3a 300 sesiones	server.py → Gateway (hilos, semáforo, plazos)	test_rs3a_300_sesiones_simultaneas	A19	SESSIONS_PEAK pico=300
RS3b Comparativa	bench/	–	–	benchmark/resumen.md
RS4a .pcap cifrado	scripts/captura.sh, trafico.py	tests de A01 y A13	A01, A13	pcap/02, analisis.txt
RS4b Auditoría	common.py → audit; server.py	todos (cada test enseña su línea)	A04–A11	logs/audit.log (4.966 líneas)

4.2 Pruebas

Hay **34 tests** de `unittest` y pasan todos: en la última ejecución (`make test`, el 7 de octubre) tardaron 52,7 s. `tests/test_funcional.py` tiene 13 sobre el alta, la inmutabilidad, el login, el logout y los roles; `tests/test_carga.py` tiene el de las 300 sesiones; y `tests/test_seguridad.py` tiene 20: uno que comprueba lo que se negocia y uno por cada ataque A01–A19, que exige que el ataque quede bloqueado. Cada test levanta su propio servidor en puertos libres. Igual que en el PAI-1, con `-v` cada test imprime qué prueba, lo esperado, el log del servidor en directo y el resultado, así que en `evidencias/logs/tests_20261007_120517.log` se ve el motivo de cada rechazo. `scripts/resumen_tests.py` saca de ese log la tabla por requisito (`resumen_tests.md`): 66 de 66 comprobaciones correctas, porque un test puede cubrir varios requisitos.

El de carga abre **300 túneles con 300 cuentas y 300 certificados distintos** y usa una barrera para que todos estén abiertos a la vez antes de cerrar. El log registra `SESSIONS_PEAK | pico=300`, 300 de 300 logins correctos, 0 errores, y el contador vuelve a 0 tras los logouts. Tardan unos 17 s en estar todos arriba, casi todo por calcular 300 PBKDF2 a la vez, no por TLS.

4.3 Ataques simulados

`ataques/ataques.py` lanza 19 ataques contra un gateway de pruebas en 127.0.0.1. Los 19 quedaron **bloqueados** (`evidencias/ataques/resultado_20261007_115502.log`, cada uno con su línea de `audit.log`).

ID	Ataque	Qué lo bloquea	Resultado / registro
A01	Sniffing del login	Cifrado TLS 1.3	Contraseña visible sin cifrar: 1 vez; con TLS: 0
A02	Downgrade a TLS 1.2, 1.1 y 1.0 (Python y `s_client`)	MinProtocol = TLSv1.3	Alerta *protocol version*; TLS_REJECT UNSUPPORTED_PROTOCOL
A03	Ofrecer solo AES-128-GCM / AES-128-CCM	Ciphersuites del .cnf	TLS_REJECT NO_SHARED_CIPHER
A04	Conectar sin certificado de cliente	CERT_REQUIRED	CERTIFICATE_REQUIRED; PEER_DID_NOT_RETURN_A_CERTIFICATE
A05	Certificado de una CA del atacante	Solo se confía en ca.crt	UNKNOWN_CA; "self-signed certificate"
A06	Usar el certificado del servidor como cliente	Extended Key Usage (serverAuth ≠ clientAuth)	UNSUPPORTED_CERTIFICATE; "unsuitable certificate purpose"
A07	Certificado de A con credenciales de B	Comprobación CN == usuario	"credenciales inválidas"; LOGIN_FAIL cn_no_coincide
A08	Fuerza bruta (10 contraseñas)	Bloqueo a los 5 fallos	La contraseña buena también se rechaza; LOCKOUT
A09	Certificado revocado	CRL (VERIFY_CRL_CHECK_LEAF)	CERTIFICATE_REVOKED en el handshake
A10	UPDATE / DELETE directo en la BD	Triggers de SQLite	"registro inmutable" en las tres órdenes
A11	JSON en texto plano contra el 7443	El puerto solo habla TLS	Sin respuesta; WRONG_VERSION_NUMBER
A12	Servidor falso (autofirmado o con cert. ajeno)	El cliente valida cadena y nombre	"posible suplantación. Conexión cancelada"
A13	MITM pasivo	Cifrado	Login correcto a través del relé; 6.052 bytes vistos, nada legible
A14	MITM activo: invierte 1 bit	Etiqueta AEAD (GCM)	BAD_RECORD_MAC; conexión abortada
A15	MITM con su propio certificado	Validación del servidor en el cliente	El cliente aborta; 0 credenciales robadas
A16	Replay de 2.865 bytes grabados	Claves ECDHE nuevas en cada conexión	Handshake falla; no se ejecuta nada
A17	Usar el túnel tras logout / reanudar	Cierre + num_tickets = 0	Envío falla; session_reused = False

A18	Comandos sin login / recurso fuera de perfil	Estado de sesión y PERFILES	"no autenticado"; ACCESS_DENIED
A19	50 handshakes a medias	Plazo total de 10 s	50/50 cerradas por timeout; el usuario legítimo entra

4.4 Análisis de las capturas (RS4a)

Las cuatro capturas están en `evidencias/pcap/` y se grabaron el 7 de octubre con `scripts/captura.sh` en la interfaz loopback. Para tener con qué comparar, `01_login_sin_cifrar.pcap` es el mismo login contra el servidor sin cifrar de `bench/` (puerto 7444). Siguiendo la secuencia TCP se lee todo:

```
tshark -r 01_login_sin_cifrar.pcap -q -z follow,tcp,ascii,0

Node 0: 127.0.0.1:35984   Node 1: 127.0.0.1:7444
{"cmd": "LOGIN", "username": "pdi_001", "password": "Seed-pdi_001-2026"}
{"ok": true, "role": "PDI"}
{"cmd": "MSG", "text": "mensaje de prueba"}
{"ok": true, "id": 195601}
{"cmd": "LOGOUT"}
{"ok": true}
```

Figura 2: secuencia TCP de 01_login_sin_cifrar.pcap; usuario y contraseña en claro

En `02_login_tls13_mtls.pcap` se hace exactamente lo mismo contra el 7443. Después del ClientHello y el ServerHello, todo son registros *Application Data*:

```
tshark -r 02_login_tls13_mtls.pcap

  4  0.000265  127.0.0.1 → 127.0.0.1  TLSv1.3  310 Client Hello (SNI=localhost)
  6  0.002827  127.0.0.1 → 127.0.0.1  TLSv1.3 3144 Server Hello, Change Cipher Spec, Application Data,
Application Data, ...
  8  0.010250  127.0.0.1 → 127.0.0.1  TLSv1.3 2526 Change Cipher Spec, Application Data, Application
Data, Application Data
 10  0.050622  127.0.0.1 → 127.0.0.1  TLSv1.3  161 Application Data
 12  0.317266  127.0.0.1 → 127.0.0.1  TLSv1.3  116 Application Data
 13  0.317417  127.0.0.1 → 127.0.0.1  TLSv1.3  132 Application Data
 15  0.317726  127.0.0.1 → 127.0.0.1  TLSv1.3  115 Application Data
 ...
 21  0.318324  127.0.0.1 → 127.0.0.1  TLSv1.3   90 Application Data
 22  0.318375  127.0.0.1 → 127.0.0.1  TCP 66 40758 → 7443 [FIN, ACK]

tls.handshake.ciphersuite = 0x1302   supported_version = 0x0304   key_share_group = 29
```

Figura 3: paquetes de 02_login_tls13_mtls.pcap y campos del ServerHello

El ServerHello (paquete 6) lleva la suite **0x1302**, que es `TLS_AES_256_GCM_SHA384`, la versión **0x0304** (TLS 1.3) y el grupo **29**, que es X25519, o sea ECDHE efímero y por tanto PFS. Los certificados ya no se ven, porque en TLS 1.3 van cifrados después del ServerHello; el tamaño de los paquetes 6 y 8 (3.144 y 2.526 bytes) se debe sobre todo al certificado del servidor y al del dispositivo. Entre el paquete 10 y el 12 pasan unos 267 ms, que es lo que tarda el servidor en calcular el PBKDF2 del login. `scripts/captura.sh` lo comprueba con `tshark` y lo deja en `analisis.txt`: la contraseña `Seed-pdi_001-2026` aparece 1 vez en la captura 01 y **0 veces** en la 02 y en la 04 (210 paquetes del MITM).

`03_ataques_downgrade_cipher.pcap` recoge los rechazos: los cuatro ClientHello de TLS 1.2, 1.1 y 1.0 reciben *Alert (Fatal, Protocol Version)* nada más llegar (paquetes 91, 101, 111 y 121), y el que solo ofrece AES-128 recibe *Handshake Failure* (paquete 131). En total hay 6 alertas.

4.5 Rendimiento con 300 usuarios (RS3b)

`bench/benchmark.py` compara el gateway con `bench/plain_server.py`, que hace el mismo trabajo (login con PBKDF2, comprobación de revocación, etc.) pero sin TLS, para que la diferencia sea solo el cifrado. Cada cliente se conecta, hace login, manda 50 mensajes de 1 KiB y cierra; repetimos cada nivel 3 veces. Se ejecutó en uno de nuestros portátiles (Intel Core i7-10750H, 12 hilos, 16 GB, Linux), con cliente y servidor en la misma máquina.

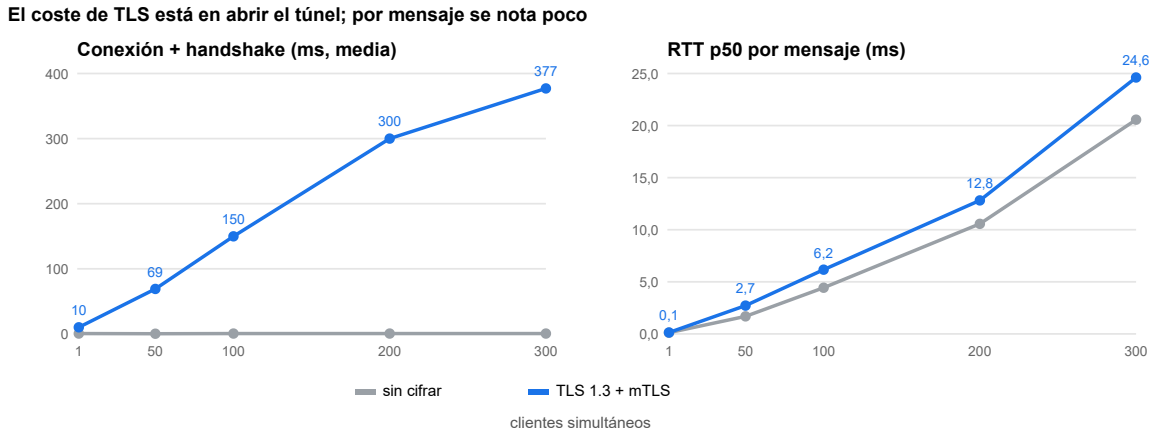


Figura 4: datos de evidencias/benchmark/resumen.md, generados con make bench

Clientes	Handshake (ms)	RTT p50 (ms)	RTT p95 (ms)	Throughput (msg/s)	CPU servidor (s)	RAM servidor (MiB)	Bytes/msg
1	0,5 / 10,2	0,11 / 0,13	0,23 / 0,24	6830 / 6944	0,25 / 0,28	26 / 28	1199 / 1369
50	0,3 / 69,1	1,68 / 2,71	7,3 / 9,2	1521 / 1672	32,4 / 34,9	30 / 37	1199 / 1369
100	0,6 / 150,0	4,43 / 6,16	19,8 / 22,1	1340 / 1367	66,9 / 70,1	33 / 46	1199 / 1369
200	0,5 / 300,1	10,57 / 12,82	46,5 / 54,1	1222 / 1017	136,7 / 140,4	41 / 61	1203 / 1373
300	0,4 / 377,0	20,57 / 24,61	108,7 / 128,6	1032 / 1098	206,9 / 210,9	58 / 78	1207 / 1376

Cada celda es *sin cifrar / TLS 1.3 + mTLS*, media de 3 repeticiones; las desviaciones están en `resumen.md`. En todas las rondas el 100 % de los clientes terminó bien.

Lo que más se nota es **abrir el túnel**: sin cifrar conectar cuesta menos de 1 ms y con TLS 10 ms con un cliente, y llega a 377 ms de media cuando entran 300 a la vez porque todos compiten por la CPU (ECDHE, firma RSA-3072 del servidor y verificación del certificado y la CRL del cliente). Una vez dentro, cada mensaje tarda entre 0,02 ms y 4 ms más (RTT p50), y el throughput es parecido en los dos casos: sale entre un 17 % peor y un 10 % mejor según el nivel, lo que entra dentro de lo que varía entre repeticiones. Por el cable pasa un 14 % más (1.369 frente a 1.199 bytes por mensaje), por la cabecera de 5 bytes y la etiqueta AEAD de 16 bytes de cada registro. La CPU del servidor sube solo un 2-8 % con 50 clientes o más porque lo que más gasta en los dos escenarios es el **PBKDF2** (unos 16 s de login con 300 clientes, con y sin TLS), y la RAM sube entre 3 y 21 MiB por el estado TLS de cada conexión. Con esto, 300 empleados a la vez se atienden sin errores en un portátil, y el cuello de botella es el hash de las contraseñas y el GIL de Python, no el cifrado.

4.6 Objetivo extra: interceptación y suplantación

Para el objetivo 5 hicimos `ataques/mitm_proxy.py`, un relé que se mete entre el cliente y el gateway de pruebas y tiene tres modos. En **modo pasivo** (A13) reenvía todo sin tocar: el login funciona, pero lo único que ve son registros TLS. Su volcado, `A13_volcado_mitm_pasivo.txt`, empieza por `16 03 01` (ClientHello) y `16 03 03` (ServerHello), y después todo son registros `17 03 03` (Application Data) de los que no se saca nada. En **modo activo** (A14) invierte un bit del tercer registro cifrado y el otro extremo lo detecta al comprobar la etiqueta GCM (`bad record mac`), corta la conexión y queda en el log. En **modo suplantación** (A15) presenta su propio certificado y el cliente se niega a seguir antes de mandar las credenciales. Con A12 (servidor falso) y A16 (replay) queda cubierto lo que pide el objetivo: el sistema resiste el ataque y además lo identifica, porque cada intento deja su línea en el `audit.log` o un mensaje claro en el cliente.

5. Limitaciones y mejoras

Limitación	Causa	Cómo se mejoraría
El contador de bloqueo está en memoria y se pierde al reiniciar	Sencillez	Guardarlo en la BD con su fecha
Un hilo por conexión	Vale para cientos de sesiones	Para miles, `asyncio` o `selectors`

Un único candado para SQLite y otro para la CA	Sencillez	Una conexión por hilo o una BD con concurrencia real
La clave de la CA está en disco sin cifrar (permisos 600)	Entorno de laboratorio	CA raíz fuera de línea, CA intermedia para firmar y clave en un HSM
La CRL caduca a los 30 días	Se regenera al arrancar	`pki.sh crl` periódico con cron, o OCSP
Certificado del servidor para `localhost`	Todo en loopback	Certificado con el nombre real del gateway
Contraseñas de las 300 cuentas en claro en un CSV	Hacen falta para los tests de carga	Solo en pruebas; en real cada usuario se da de alta con su código
El benchmark es en loopback	Una sola máquina	Medir entre dos máquinas; con latencia real el peso relativo de TLS sería menor

6. Uso de inteligencia artificial

Para el desarrollo de la práctica nos hemos apoyado en herramientas de Inteligencia Artificial generativa:

Herramientas utilizadas: Claude Opus 5.5 vía Claude Code

Tareas en las que se usó: igual que en el PAI-1, hemos usado Claude para la generación de código: la estructura del repositorio, el servidor y el cliente, los scripts de la PKI, los tests, los ataques y el benchmark, y la depuración. También nos ayudó a plantear la estructura de la memoria, con la corrección ortográfica y con las figuras de apoyo (el diagrama de secuencia y la gráfica).

Revisión y verificación por parte del grupo: la parte humana ha consistido en comprobar manualmente:

- Que el código y los tests funcionan, ejecutándolos nosotros y leyendo los logs.
- Que los ataques se ejecutan de verdad y que el motivo del bloqueo es el que esperamos, mirando las capturas en Wireshark.
- Que se cumplen los objetivos y los requisitos del enunciado, uno por uno, con la matriz del apartado 4.1.
- La corrección de fallos y alucinaciones de la IA.
- El estudio de TLS 1.3, mTLS, X.509 y la PKI para entender lo que hace el código y poder defenderlo.

Limitaciones observadas: la primera versión restringía las suites con `set_ciphers()` y daba por hecho que así solo se negociaban las fuertes, pero al probar el ataque de suite débil vimos que el servidor seguía aceptando `TLS_AES_128_GCM_SHA256`; de ahí salió el fichero `openssl_tls13.cnf`. Tampoco tenía en cuenta que OpenSSL 3.5 y posteriores negocian por defecto grupos híbridos post-cuánticos, que no hemos visto en clase, así que los dejamos fuera. Y en la revisión de seguridad salió que el bloqueo por fuerza bruta se podía saltar con varios login en paralelo, porque el intento se contaba después del PBKDF2. Todo esto lo corregimos en varias iteraciones.

7. Conclusiones

Con esta práctica hemos pasado de proteger los mensajes a mano, como en el PAI-1, a apoyarnos en TLS 1.3 y en una PKI. Lo que más nos ha costado no ha sido activar TLS, que es poco código, sino asegurarnos de que se aplica lo que creemos que se aplica: que de verdad solo se negocian las dos suites que queremos, que la CRL se recarga, que una revocación corta las sesiones abiertas o que el bloqueo no se puede esquivar. Para eso nos han servido mucho los ataques automáticos, porque cada uno nos obligaba a comprobar la defensa con el tráfico real y no solo leyendo el código.

El resultado cumple los cuatro objetivos y el extra: los 34 tests pasan, los 19 ataques quedan bloqueados, el servidor aguanta 300 sesiones a la vez y las capturas demuestran que ni la contraseña ni los datos se ven en el cable. También hemos visto que el coste de TLS está casi todo en el handshake y que, una vez abierto el túnel, la diferencia con el tráfico sin cifrar es pequeña. Lo que más pesa con 300 usuarios es el PBKDF2, que es un coste que asumimos a propósito para proteger las contraseñas.

Lo que nos queda por mejorar está en el apartado 5, y casi todo es lo que haría falta para pasar del laboratorio a un despliegue real: proteger mejor la clave de la CA, un certificado de servidor con su nombre real y medir entre máquinas distintas.

8. Manual de despliegue y uso

8.1 Qué hace el proyecto

El proyecto es una VPN *road warrior* para la Universidad U-Secure: un empleado se conecta desde su portátil personal, por ejemplo desde el Wi-Fi de una cafetería, y llega a los servicios de la universidad (correo, docencia, gestión…) por un túnel cifrado con **TLS 1.3**. Hay dos programas:

- **El gateway** (`src/server.py`), que hace de servidor VPN de la universidad.
- **El cliente** (`src/client.py`), que hace de portátil del empleado.

Para entrar hacen falta **dos cosas a la vez**: el **certificado del dispositivo**, firmado por la CA de la universidad (sin él el servidor corta la conexión en el handshake), y el **usuario y la contraseña**. El gateway escucha en dos puertos:

Puerto	Para qué	Seguridad
7443	El túnel: login, mensajes, recursos y logout	TLS 1.3 + mTLS: el servidor exige certificado de cliente válido y no revocado
8443	Solo dar de alta un dispositivo nuevo (`REGISTER`)	TLS 1.3 autenticando solo al servidor, porque el dispositivo todavía no tiene certificado

Todo está hecho con la librería estándar de Python y la herramienta `openssl`, así que no hay que instalar ningún paquete de Python. Los recursos de la universidad están simulados: devuelven un texto fijo.

8.2 Requisitos e instalación

Funciona en cualquier distribución de Linux (Ubuntu, Debian, Fedora…). Hace falta lo siguiente:

Herramienta	Versión probada	Para qué
Python	3.14.7 (vale ≥ 3.10)	Servidor, cliente, tests, ataques y benchmark
OpenSSL	3.6.4 (vale ≥ 3)	TLS y la CA (`pki/pki.sh`)
make	cualquiera	Todos los comandos del proyecto pasan por el `Makefile`
tcpdump y tshark	-	Solo para `make captura` (necesita `sudo`)

Python, OpenSSL y make vienen instalados en casi todas las distribuciones. Si falta algo, se instala con el gestor de paquetes:

```
Terminal
# Ubuntu / Debian
$ sudo apt update
$ sudo apt install python3 openssl make tcpdump tshark
# Fedora
$ sudo dnf install python3 openssl make tcpdump wireshark-cli
```

Durante la instalación de `tshark`, Ubuntu pregunta si los usuarios sin privilegios pueden capturar; da igual la respuesta, porque `make captura` ya usa `sudo`. Para saber qué versión de Python hay: `python3 --version` (Ubuntu 22.04 o posterior ya trae 3.10 o superior).

Se descomprime `PAI2-ST5.zip` (o se clona el repositorio), se entra en la carpeta y se comprueba que está todo:

```
Terminal
$ cd PAI2-ST5
$ make check
Python 3.14.7
OpenSSL 3.6.4 25 Aug 2026
✓ openssl
✓ tcpdump
✓ tshark
✓ ulimit -n = 1048576
```

Figura 5: salida de make check con todo instalado

Si `ulimit -n` sale por debajo de 1024, hay que ejecutar `ulimit -n 4096` en esa terminal; si no, con 300 clientes a la vez se acaban los descriptores de fichero.

8.3 Puesta en marcha

8.3.1 Crear la CA y los usuarios de prueba

Solo hace falta una vez:

```
Terminal
$ make seed
```

Tarda unos 30 segundos y hace dos cosas:

1. Crea la **CA de la universidad** ("U-Secure Root CA", RSA 3072), el certificado del servidor y una CRL vacía, todo en `pki/`.
2. Da de alta **300 usuarios de prueba** con su certificado (RF5): `pdi_001`…`pdi_100`, `pas_001`…`pas_100` e `inv_001`…`inv_100`. Sus contraseñas están en `pki/seed_credentials.csv` y siguen el patrón `Seed-<usuario>-2026`; por ejemplo, la de `pdi_001` es `Seed-pdi_001-2026`. Cada "portátil" queda en `clientes/<usuario>/` con su `device.key` y su `device.crt`.

Si se vuelve a ejecutar no duplica nada. Para empezar de cero: `make clean && make seed`.

8.3.2 Arrancar el servidor y el cliente

Hacen falta dos terminales dentro de la carpeta del proyecto, y una tercera si se quiere ver el registro en directo:

Terminal	Comando	Qué es
1	`make server`	El gateway. Se deja abierto; se para con Ctrl+C
2	`make client`	El menú del cliente
3 (opcional)	`tail -f evidencias/logs/audit.log`	El registro de auditoría en directo

Al arrancar, el servidor enseña la política TLS que está aplicando de verdad. Si no sale esto, algo va mal (ver el apartado 8.8):

```
Terminal 1 · make server
Política TLS efectiva: TLSv1_3..TLSv1_3 · TLS_AES_256_GCM_SHA384,
                       TLS_CHACHA20_POLY1305_SHA256 · grupos X25519:secp384r1
Gateway VPN escuchando en :7443 (túnel) y :8443 (alta). Ctrl+C para parar.
```

Figura 6: arranque del gateway

8.4 Uso del cliente

En la terminal 2 aparece el menú:

```
Terminal 2 · make client
--- U-Secure VPN ---  sin sesión
1) Registrar dispositivo  2) Iniciar sesión  3) Enviar mensaje
4) Acceder a recurso      5) Cerrar sesión   0) Salir
>
```

Figura 7: menú del cliente

8.4.1 Iniciar sesión con un usuario precargado

1. Opción **2**, usuario `pdi_001` y contraseña `Seed-pdi_001-2026`. La contraseña no se ve al escribirla (se pide con `getpass`). Sale ✓ `Túnel establecido: TLSv1.3 · TLS_AES_256_GCM_SHA384 · servidor:`

`localhost · rol PDI`.

2. Opción **4** y `docencia`: devuelve el contenido simulado.
3. Opción **4** y `gestion`: `ACCESS_DENIED`, porque un PDI no tiene acceso a la gestión administrativa.
4. Opción **3** y un texto: se guarda en la base de datos y devuelve `{'ok': True, 'id': …}`. El texto no se escribe en el log.
5. Opción **5**: `Sesión cerrada y túnel destruido.` El cliente manda `LOGOUT`, los dos extremos cierran TLS y el socket, y como el servidor no emite tickets la sesión no se puede reanudar (RF4).

Qué recursos puede pedir cada rol:

Rol	correo	docencia	gestion	repositorio	bd_investigacion	servidores_internos
PDI	Sí	Sí	No	Sí	Sí	No
PAS	Sí	No	Sí	Sí	No	No
INV	Sí	No	No	Sí	Sí	Sí

8.4.2 Dar de alta un usuario nuevo (RF1)

Un usuario no puede registrarse por su cuenta: primero tiene que autorizarlo el administrador.

1. **Administrador**, con el servidor en marcha y desde otra terminal: `python3 src/server.py alta ana_garcia PDI`. Devuelve un código de un solo uso, por ejemplo `hLoHBSN3X5rGP-ZvMO1-BA`. El usuario debe tener entre 3 y 32 caracteres (minúsculas, números o _) y el rol tiene que ser `PDI`, `PAS` o `INV`.
2. **Usuario**, en el menú del cliente: opción **1**, usuario `ana_garcia`, una contraseña de al menos 8 caracteres y el código. El cliente genera la clave privada en el propio equipo (no sale nunca de él), manda una CSR por el puerto 8443 y la CA la firma.

```
Terminal 2 · opción 1

✓ Túnel seguro con el servidor: TLSv1.3 · TLS_AES_256_GCM_SHA384 · servidor: localhost
✓ Dispositivo registrado como PDI. Ya puedes iniciar sesión.
```

Figura 8: alta correcta de un dispositivo

La clave y el certificado se guardan en `clientes/ana_garcia/`. El código solo vale una vez, y una vez hecha el alta los datos del usuario y su certificado no se pueden modificar en la base de datos (los triggers lo impiden).

8.4.3 Revocar un dispositivo (portátil robado)

```
Terminal

$ python3 src/server.py revocar ana_garcia
✓ Certificado 114F de ana_garcia revocado; el gateway recarga la CRL solo.
```

No hace falta reiniciar el servidor: recarga la CRL al ver que ha cambiado. Si `ana_garcia` tenía una sesión abierta, se le corta en el siguiente comando, y si intenta entrar otra vez el handshake falla y el cliente dice `✗ Tu certificado ha sido revocado. Pide al administrador un alta nueva.` Para devolverle el acceso se repite el apartado 8.4.2 con la misma contraseña de antes.

8.4.4 Modo por comandos

Para scripts o para no pasar por el menú, el cliente acepta órdenes sueltas. Si no se pone `--password`, la pide sin mostrarla:

```
Terminal

$ python3 src/client.py login    --user pdi_001
$ python3 src/client.py get      --user pas_001 --recurso gestion
$ python3 src/client.py msg      --user inv_001 --texto "hola"
$ python3 src/client.py register --user ana_garcia --code <código>
```

8.5 Registro de auditoría

El servidor escribe una línea por evento en `evidencias/logs/audit.log`, con la fecha en UTC, la IP y el puerto de origen, el usuario, el CN y número de serie del certificado, la versión de TLS y la suite. Nunca aparecen contraseñas, códigos de alta ni el texto de los mensajes.

evidencias/logs/audit.log

```
… | LOGIN_OK    | 127.0.0.1:33326 | user=pdi_001 | rol=PDI | cn=pdi_001 | serial=1003
            | tls=TLSv1.3 | suite=TLS_AES_256_GCM_SHA384
… | TLS_REJECT | 127.0.0.1:33340 | motivo=PEER_DID_NOT_RETURN_A_CERTIFICATE
```

Figura 9: un login correcto y una conexión sin certificado

Evento	Cuándo aparece
SERVER_START, CRL_RELOAD	Al arrancar (con la política TLS efectiva) y cada vez que cambia la CRL
CONNECT, TLS_OK, DISCONNECT	Conexión TCP, handshake correcto y cierre
TLS_REJECT	Handshake rechazado: versión antigua, suite débil, sin certificado, certificado revocado o de otra CA…
LOGIN_OK, LOGIN_FAIL, LOCKOUT	Login correcto, fallo con su motivo real y bloqueo tras 5 fallos (5 min)
MSG, ACCESS_DENIED, UNAUTH_CMD	Mensaje guardado, recurso fuera del perfil y comando sin haber hecho login
LOGOUT	Cierre de sesión pedido por el usuario
ENROL_CODE, ENROL_OK, ENROL_FAIL	Código de alta generado y alta correcta o fallida
REVOKE, SESSION_REVOKED	Certificado revocado y sesión cortada por esa revocación
SESSIONS_PEAK, CAPACITY_REJECT	Nuevo máximo de sesiones simultáneas y conexión rechazada por superar el tope (400)

8.6 Comprobar la seguridad a mano

Con `make server` en marcha se pueden probar estos casos y ver la respuesta y la línea del `audit.log`:

Qué se prueba	Cómo	Qué tiene que pasar
Fuerza bruta	Opción 2 con `pdi_002` y una contraseña inventada, 5 veces	La sexta vez ni la buena entra durante 5 min; `LOCKOUT`
Recurso fuera de perfil	`pas_001` pide `bd_investigacion`	`ACCESS_DENIED`
Forzar TLS 1.2	`openssl s_client -connect 127.0.0.1:7443 -tls1_2 -CAfile pki/ca/ca.crt`	`alert protocol version`; TLS_REJECT UNSUPPORTED_PROTOCOL
Suite débil	`openssl s_client -connect 127.0.0.1:7443 -ciphersuites TLS_AES_128_GCM_SHA256 -CAfile pki/ca/ca.crt`	`alert handshake failure`; TLS_REJECT NO_SHARED_CIPHER
Sin certificado	`openssl s_client -connect 127.0.0.1:7443 -CAfile pki/ca/ca.crt`	`alert certificate required` al escribir algo
Conexión buena por dentro	`openssl s_client -connect 127.0.0.1:7443 -CAfile pki/ca/ca.crt -cert clientes/pdi_001/device.crt -key clientes/pdi_001/device.key`	TLSv1.3, TLS_AES_256_GCM_SHA384 y grupo X25519

En la última prueba, una vez conectado se puede hablar el protocolo a mano escribiendo una línea JSON y pulsando Intro: `{"cmd":"LOGIN","username":"pdi_001","password":"Seed-pdi_001-2026"}`, luego `{"cmd":"GET","resource":"docencia"}` y por último `{"cmd":"LOGOUT"}`.

8.7 Pruebas automáticas y evidencias

Antes de `make bench` y `make captura` hay que parar el servidor manual (Ctrl+C), porque usan los mismos puertos. `make test` y `make ataques` levantan su propio servidor en puertos libres y no chocan.

Comando	Tarda	Qué hace	Dónde está el resultado
`make test`	~1 min	34 tests: 13 funcionales, 300 sesiones simultáneas y los 19 ataques	`evidencias/logs/tests_<fecha>.log` y `resumen_tests.md`
`make ataques`	~30 s	Los 19 ataques explicados uno a uno; tiene que acabar en 19/19 bloqueados	`evidencias/ataques/resultado_<fecha>.log`
`make bench`	~7 min	TLS frente a sin cifrar con 1, 50, 100, 200 y 300 clientes, 3 veces cada uno	`evidencias/benchmark/resumen.md` y `resultados.csv`

`make captura`	~1 min	Graba 4 capturas .pcap y comprueba con tshark que la contraseña no se ve. Pide la contraseña de `sudo` solo para tcpdump (no lanzarlo con `sudo make`)	`evidencias/pcap/*.pcap` y `analisis.txt`
`make evidencias`	~10 min	Todo lo anterior desde cero (borra y regenera)	toda la carpeta `evidencias/`

Cada test imprime qué prueba, qué espera, el log del servidor en directo y si pasa o no; al final tiene que salir `Ran 34 tests … OK`. Para ver las capturas como las vería un espía en la red:

Terminal

```
# sin cifrar: la contraseña se ve
$ tshark -r evidencias/pcap/01_login_sin_cifrar.pcap -Y 'frame contains "Seed-"'
# con TLS: solo "Application Data"
$ tshark -r evidencias/pcap/02_login_tls13_mtls.pcap -d tcp.port==7443,tls
```

También se pueden abrir en Wireshark con el filtro `tcp.port == 7443` y la opción Seguir → Secuencia TCP.

8.8 Problemas frecuentes

Lo que sale	Por qué	Qué hacer
El gateway VPN no responde: ¿está arrancado?	El servidor no está en marcha	`make server` en otra terminal
Address already in use	Ya hay un servidor (o `make bench`) usando el puerto	Pararlo con Ctrl+C o `pkill -f src/server.py`
Este dispositivo no tiene certificado	Ese usuario no se ha registrado en este equipo	Opción 1 del menú con un código de alta
credenciales inválidas	Contraseña mala, o certificado de otro usuario	Revisar el usuario; el motivo real está en `audit.log`
Tu certificado ha sido revocado	El administrador lo revocó	Pedir un alta nueva (apartado 8.4.2)
Política TLS no cargada	Algún módulo cargó OpenSSL antes que `common.py`	No cambiar el orden de los `import`; `common` va antes que `ssl` y `hashlib`
Too many open files	Límite de descriptores bajo	`ulimit -n 4096`
Los tests dicen "falta ejecutar make seed"	No hay usuarios precargados	`make seed`
make captura falla	Faltan `tcpdump`/`tshark` o no se hizo `make seed`	Instalarlos y repetir

8.9 Resumen de comandos

Comando	Para qué
`make check`	Comprobar que está todo instalado
`make seed`	CA + 300 usuarios (pdi_001 / Seed-pdi_001-2026)
`make server`	Arrancar el gateway (terminal 1)
`make client`	Menú del cliente (terminal 2)
`tail -f evidencias/logs/audit.log`	Registro en directo (terminal 3)
`python3 src/server.py alta <usuario> <PDI|PAS|INV>`	Autorizar un alta
`python3 src/server.py revocar <usuario>`	Revocar un dispositivo
`make test` · `make ataques` · `make bench` · `make captura`	Regenerar cada tipo de evidencia
`make evidencias`	Regenerar todas las evidencias desde cero
`make zip`	Crear el zip de entrega (sin claves privadas ni `__pycache__`)
`make clean`	Borrar todo lo generado (CA, usuarios, BD y evidencias)

9. Anexo

A.1 Estructura del proyecto

```
PAI2-ST5/
├── Makefile                Punto de entrada: check, seed, server, client, test, ataques, bench, captura...
├── openssl_tls13.cnf       Política TLS: solo TLS 1.3, dos suites AEAD, grupos X25519 y secp384r1
├── src/
│   ├── common.py           Contextos TLS, protocolo JSON y auditoría (se importa antes que ssl)
│   ├── server.py           Gateway VPN (:7443 túnel mTLS, :8443 alta) y comandos alta/revocar
│   ├── altas.py            Alta y revocación (lo usan el servidor y seed.py)
│   ├── db.py               SQLite con triggers de inmutabilidad y PBKDF2
│   ├── client.py           Cliente BYOD: menú y modo por comandos
│   └── seed.py             300 cuentas PDI/PAS/INV con certificado
├── pki/
│   ├── pki.sh              CA corporativa: init, server, sign, revoke, crl
│   └── openssl_ca.cnf      Perfiles de certificado (serverAuth / clientAuth)
├── tests/
│   ├── base.py             Servidor de pruebas en puertos libres y cabecera de cada test
│   ├── test_funcional.py   Alta, inmutabilidad, doble autenticación, logout y roles (13)
│   ├── test_carga.py       300 sesiones simultáneas (1)
│   └── test_seguridad.py   Negociación + los 19 ataques (20)
├── ataques/
│   ├── ataques.py          A01-A19 contra el gateway de pruebas en 127.0.0.1
│   └── mitm_proxy.py       Relé MITM: pasivo, altera bytes o suplanta
├── bench/
│   ├── plain_server.py     El mismo servidor sin cifrar (solo para comparar)
│   └── benchmark.py        TLS frente a sin cifrar con 1, 50, 100, 200 y 300 clientes
├── scripts/                captura.sh (tcpdump + tshark), trafico.py, resumen_tests.py
├── docs/                   PAI2-ST5.pdf (esta memoria)
└── evidencias/
    ├── logs/               audit.log, tests_*.log, resumen_tests.md
    ├── ataques/            resultado_*.log, A13_volcado_mitm_pasivo.txt
    ├── benchmark/          resultados.csv, resumen.md
    └── pcap/               01 a 04 .pcap y analisis.txt
```

Figura 10: Estructura del proyecto